

LAMG+: A FASTER, MORE ROBUST LEAN ALGEBRAIC MULTIGRID SOLVER FOR GRAPH LAPLACIANS*

OREN E. LIVNE†

Abstract. Graph-Laplacian systems $L\phi = b$ underlie spectral clustering, centrality, semi-supervised learning, finite-element analysis, and interior-point network-flow solvers. We present **LAMG+**, a lean, parameter-free, empirically linear-time algebraic multigrid solver: a Julia re-derivation of Lean Algebraic Multigrid (LAMG) plus two targeted refinements. We establish three facts. (1) *Fair, same-machine benchmarking* against both approximate-Cholesky (AC) variants—the modern near-linear champion—and four other state-of-the-art solvers (hypre/BoomerAMG, PETSc GAMG, pyAMG, CMG), all run first-hand. LAMG+ and AC are *complementary peers*: AC is faster on social and citation graphs, LAMG+ on finite-element/structural matrices (where it is the fastest robust solver and the most memory-frugal) and $2.2\times$ faster than the worst-case-robust AC variant on large graphs; across all twelve test classes only LAMG+ and AC are never catastrophic, while BoomerAMG, PETSc, pyAMG, and CMG each slow by an order of magnitude or fail to converge off their home turf. (2) *Linear scaling*: LAMG+ is empirically $\mathcal{O}(m)$ over the full 1,711-graph SuiteSparse LAMG set (100% converged, median 4 cycles, log-log slope 1.01), verified to 2.4×10^8 nonzeros (**com-0rkut**, twice the original LAMG study), and—in the same implementation— $\sim 2\times$ faster in the geometric mean than the LAMG it re-derives (up to $39\times$ on grid-aligned-anisotropic grids where LAMG’s convergence collapses): the sense in which it is “faster.” (3) *Robustness*: the original MATLAB LAMG was reported non-convergent across the AC benchmark’s test families, whereas our re-derivation converges to their 10^{-8} tolerance on all of them, indicating the gap is specific to that implementation, not the algorithm. The two refinements, a strength-of-connection aggregation veto and a selective caliber-2 interpolation, close LAMG’s grid-aligned-anisotropy outlier tail (asymptotic convergence factor $\approx 0.99 \rightarrow 0.11$). Both trigger locally and do not substantially increase the complexity. Being a multigrid approach rather than a factorization, LAMG+’s hierarchy is readily reused within a nonlinear network-flow convex equilibrium solver (which in particular solves max-flow) in a companion paper.

Key words. graph Laplacian, algebraic multigrid, aggregation, piecewise-constant interpolation, linear-time solver, approximate Cholesky, adaptive multigrid

AMS subject classifications. 65F08, 65F10, 65N55, 05C50, 90C35

1. Introduction. Let $G = (V, E, w)$ be a connected weighted undirected graph with $|V| = n$ nodes, $|E| = m$ edges, and positive weights $w : E \rightarrow \mathbb{R}^+$. Its Laplacian $L = D - W$ (W the weighted adjacency, D the weighted-degree diagonal) is symmetric positive semidefinite with the constant vector $\mathbf{1}$ spanning its null space; equivalently $\phi^\top L\phi = \sum_{(u,v) \in E} w_{uv}(\phi_u - \phi_v)^2$. We solve the compatible system

$$(1.1) \quad L\phi = b, \quad \mathbf{1}^\top \phi = 0, \quad \mathbf{1}^\top b = 0.$$

Typically $m \ll n^2$ and L is sparse. Our goal is an iterative solver for (1.1) that uses $\mathcal{O}(m)$ storage and $\mathcal{O}(m \log(1/\varepsilon))$ operations for an ε -accurate solution, with *bounded hidden constants on the graphs that arise in applications*—hundreds, not millions. As in the original LAMG work [1], we target good *empirical* performance over a diverse test set rather than provable worst-case complexity on adversarial graphs; in practice the latter bounds are loose or absent on the hidden constants that govern wall-clock time [2].

1.1. Applications. System (1.1) is fundamental to many computations [2, §2]: elliptic PDEs discretised by finite elements on unstructured grids [19]; interior-point

*Submitted to the editors June 17, 2026.

†Pine Birch Analytics, 35 Kelinger Rd, Churchville, PA 18966-1033 (oren.livne@gmail.com, tel. 312-533-9130, pinebirchanalytics.com; ORCID: 0000-0001-6700-483X).

methods for network-flow linear programming, whose inner solve is a weighted Laplacian [20]; electrical flow through a resistor network; spectral clustering, graph embedding, and ancestry discovery in machine learning [21]; and the Fiedler eigenvector, which measures algebraic connectivity and underlies graph partitioning [22]. Solving $L\phi = b$ is also a stepping stone to the Laplacian eigenproblem, and—through a smooth nonlinear reformulation—to network flow itself: a saturating-resistor model recasts max-flow as a sequence of weighted-Laplacian solves, each handled by LAMG+. We plan to develop that extension in a companion paper and return to it in §6.

1.2. Related work. *Direct methods.* A Cholesky factorisation $P^\top LP = \hat{L}\hat{L}^\top$ under a fill-reducing ordering (minimum-degree, nested dissection) is exact and, with a good ordering, very fast [15, 16]. But fill is governed by separator quality: $\mathcal{O}(n^{1.5})$ work on planar graphs [18], $\mathcal{O}(n^3)$ in general, and—on low-diameter social and citation graphs with no small separators—a supernodal factorisation that exhausts memory before it finishes.

Graph-theoretic iterative methods. Spielman and Teng [3] and successors [4] build spectral sparsifiers as preconditioners, achieving for any SDD system a provably near-linear $\mathcal{O}(m \text{polylog } n \cdot \log(1/\varepsilon))$ bound. The hidden constants are large; the fast practical realisation is the approximate-Cholesky solver `approxChol` [4, 5], recently made substantially more robust by Gao, Kyng, and Spielman [23] (the `approxchol_lap2` variant we benchmark), and paralleled by randomized-Cholesky direct solvers such as `RCHOL` [28]. We benchmark against both `approxChol` variants as our strongest near-linear baselines (§4.3).

Algebraic multigrid. AMG [7, 9] builds a hierarchy of coarser operators from matrix entries alone. Classical AMG selects a coarse node subset; aggregation AMG [10, 11] groups fine nodes into aggregates. AMG excels on discretised PDEs but its complexity is hard to control on general graphs: coarse operators densify on scale-free hubs, and higher-order interpolation makes this worse. A line of work targets graph Laplacians specifically: Napov and Notay’s degree-aware rooted aggregation with aggregate-quality control [25], matching-based multilevel preconditioners [26], and effective-resistance/diffusion-distance coarsening with least-squares interpolation from relaxed vectors [27]—the last closely related to LAMG’s affinity. Bootstrap AMG [12] and algebraic distance [13] likewise learn the coarsening from relaxed test vectors, at higher setup cost; a recent direction learns the prolongation itself with graph neural networks [31]. LAMG+, by contrast, is parameter-free and training-free. Combinatorial Multigrid (CMG) [6] also targets general topologies; CMG and LAMG had comparable average speed but LAMG was markedly more robust [1]. General-purpose aggregation packages (PyAMG [29], hypre/BoomerAMG [30]) are tuned for PDE discretisations rather than the irregular degree distributions of graph Laplacians. In the broadest recent comparison [23], CMG, BoomerAMG, and PETSc all failed to converge across the chimera and SDDM families, leaving the approximate-elimination solver AC (the `approxchol_lap2` variant) the only blackbox method robust across all of them; the original MATLAB LAMG was reported non-convergent there, whereas our independent re-derivation converges on those families (§4.4)—a difference specific to that implementation, not the algorithm. We take AC as our sole near-linear baseline, with `CHOLMOD` for the direct-method crossover.

1.3. Contribution. LAMG [1] is an accelerated caliber-1 aggregation AMG that attains $\mathcal{O}(m)$ on general graphs—more precisely $\mathcal{O}(m \log(1/\varepsilon))$ for an ε -accurate solution—by regulating the coarse-operator *energy* rather than its size. We present

LAMG+: an open re-derivation of the algorithm¹ (validated line-by-line against the original MATLAB reference, App. C) that we use to establish three things.

- *A fair, same-language adjudication against the modern near-linear champion* (§4.3). LAMG (2012) predates and was never compared with Spielman’s approximate-Cholesky line [5, 23], now the dominant practical near-linear solver. Run in the same Julia process, the two are *complementary peers*: approximate-Cholesky is faster on social/citation graphs, LAMG+ on FE/structural matrices (the fastest robust solver there, 2.2× over the worst-case-robust AC variant on large graphs); the split is algorithmic (coarsening vs. fill), not a compiler artifact.
- *Linear scaling at scale* (§4.2). LAMG+ is empirically $\mathcal{O}(m)$ over 1,711 SuiteSparse graphs—100% converged, median 4 cycles, log-log slope 1.01—and builds its multilevel hierarchy on a 2.3×10^8 -edge graph, two orders of magnitude beyond LAMG’s original demonstration, with no tuning.
- *Robustness, closing LAMG’s outlier tail* (§3, §4.4). Where the original MATLAB LAMG was reported non-convergent across the benchmark’s test families, our independent re-derivation converges to their 10^{-8} tolerance on all of them—the gap specific to that implementation, not the algorithm. Two lean, *local* refinements—a strength-of-connection veto and a selective caliber-2 interpolation, both *anticipated as future work* in [1]—turn LAMG’s stalled grid-aligned-anisotropy rate (ACF ≈ 0.99) into ≈ 0.11 at $\approx 1\%$ median overhead, each acting only on the locally anisotropic nodes that need it. The ideas are LAMG’s; the correct lean realisation and the at-scale evidence are ours.
- *A forensic account of LAMG’s load-bearing design choices* (§2, §4.7). Several of LAMG’s setup choices—no aggregate-size cap, orphan nodes left as singleton seeds, a directional energy guard—are individually *load-bearing* yet undocumented as essential; we isolate them and quantify, by single-choice ablation, that replacing any one by a plausible correlate silently destroys convergence on a whole graph class (web ACF $0.002 \rightarrow 0.97$, for one). Here reproduction *is* discovery: knowing which heuristics carry the algorithm, and by how much, is what makes it transferable beyond the original implementation.

The open implementation makes all of the above reproducible; LAMG+ needs no parameter tuning.

2. The LAMG algorithm. We summarise LAMG [1] in enough detail to make this paper self-contained; the refinements of §3 presuppose it.

Multigrid in one paragraph. A relaxation such as Gauss–Seidel (GS) applied to $L\phi = b$ removes the high-residual (oscillatory) error in a few sweeps but stalls on the *algebraically smooth* error—the slow, low-energy modes. Multigrid corrects those on a coarser graph: it represents the fine smooth error $e \approx Pe^c$ by interpolation $P \in \mathbb{R}^{n \times n_c}$ from a coarse vector e^c , solves the Galerkin coarse system $L^c e^c = P^\top (b - L\phi)$ with $L^c = P^\top LP$, and corrects $\tilde{\phi} \leftarrow \tilde{\phi} + Pe^c$. Applied recursively over $\ell = 1, \dots, L$ levels this is a multigrid cycle; the work is geometric in the per-level coarsening ratio. LAMG’s two coarsening operators—elimination and aggregation—are alternated until the coarsest level is small.

Relaxation. LAMG relaxes with Gauss–Seidel, $\phi_u \leftarrow (b_u - \sum_{v \neq u} L_{uv}\phi_v)/L_{uu}$, an effective smoother for SPD systems that needs no damping parameter.

¹Open-source Julia implementation, including all scripts to reproduce the experiments in this paper: <https://github.com/orenlivne/lamgplus>.

Low-degree elimination. Let $F \subseteq V$ be a maximal independent set of nodes of degree $\leq d_{\max} = 4$. Because F is independent, L_{FF} is diagonal and the Schur complement onto $C = V \setminus F$ is *exact*:

$$(2.1) \quad L^c = L_{CC} - L_{CF}L_{FF}^{-1}L_{FC}, \quad \phi_F = P_F\phi_C + L_{FF}^{-1}b_F, \quad P_F = -L_{FF}^{-1}L_{FC}.$$

Eliminating an F -node of degree ≤ 3 does not increase m ; the rare degree-4 node may add an edge. Elimination removes the effectively-one-dimensional part of the graph at zero approximation cost (up to fill), shrinking n before each aggregation step.

Caliber-1 aggregation. Each remaining node is assigned to exactly one aggregate, giving a piecewise-constant (“caliber-1”) interpolation with $P_{u,A(u)} = 1$. This keeps $L^c = P^\top LP$ as sparse as the graph—it cannot manufacture the dense coarse rows that sink classical AMG on hubs. Aggregates are formed from K test vectors $X = (x^{(1)}, \dots, x^{(K)})$, each ν GS sweeps of $Lx = 0$ from a random start: these are samples of the algebraically smooth subspace the coarse grid must capture, with no geometry assumed. (LAMG uses $K = 4$ at the finest level, raises K by one per coarser level to a cap of 10, and smooths each by $\nu = 3$ sweeps [1, §4.1].) The proximity of an edge (u, v) is the *affinity*, the squared cosine of the two test-vector profiles [1, Eq. 3.3],

$$(2.2) \quad c_{uv} = \frac{(\sum_k x_u^{(k)} x_v^{(k)})^2}{(\sum_k (x_u^{(k)})^2)(\sum_k (x_v^{(k)})^2)} \in [0, 1].$$

A node u joins the seed s of maximal affinity that is *admissible* under the *energy-ratio guard*. With the nodal energy $E_u(x; y) = \frac{1}{2}L_{uu}y^2 - B_u(x)y + C_u(x)$, $B_u = \sum_v w_{uv}x_v$, $C_u = \frac{1}{2}\sum_v w_{uv}x_v^2$,

$$(2.3) \quad q_{u \leftarrow s} = \max_{1 \leq k \leq K} \frac{E_u(x^{(k)}; x_s^{(k)})}{\min_y E_u(x^{(k)}; y)} \leq Q, \quad Q = 2.5.$$

The numerator is u ’s energy when caliber-1 *forces* $u = x_s$; the denominator is its relaxed (GS-optimal) energy $\min_y E_u = E_u(x; B_u/L_{uu})$. Their ratio is the factor by which lumping u into s ’s aggregate inflates the local energy; capping it bounds the coarse-operator energy aggregate by aggregate, hence the two-level factor $\rho \lesssim 1 - 1/Q$ [1, §3.4]. Crucially the guard regulates *energy, not size*—there is *no* hard aggregate-size cap: the same Q admits a 100-node hub aggregate (neighbours agree on the slow modes, $q \approx 1$) yet forbids a 3-node mesh aggregate (neighbours disagree, q shoots up). Two companion conventions complete the rule and are equally load-bearing: a node that finds no admissible seed stays a singleton seed (an *orphan*), never force-absorbed; and the guard is *directional*—(2.3) scores u forced onto seed s , $q_{u \leftarrow s}$, and is not symmetrised in (u, s) . These are LAMG’s choices [1, §3.4]; substituting a plausible correlate for any of them (a size cap, forced absorption, a symmetric guard) silently over- or under-coarsens some graph class, so §3 concerns only where LAMG+*changes* LAMG. The guard dominates setup cost and is evaluated in $\mathcal{O}(m)$ via three bulk-precomputed moments per node (App. B).

Cycle and acceleration. The solve runs a γ -cycle whose per-level index LAMG fixes by a work rule [1, Eq. 4.1]: elimination levels use $\gamma_\ell = 1$, and an aggregation level uses

$$(2.4) \quad \gamma_\ell = \begin{cases} 1.5, & |E_\ell| > 0.1 |E| \text{ (finer levels),} \\ \min\{2, 0.7/\tau_\ell\}, & \text{otherwise (coarser levels),} \end{cases}$$

where $\tau_\ell = |E_{\ell+1}|/|E_\ell|$ is the level- ℓ coarsening ratio. The cap bounds per-cycle work geometrically, guaranteeing $\mathcal{O}(m)$. The coarse-level branch raises γ_ℓ toward 2 where coarsening is aggressive but lowers it toward 1 where coarsening is weak ($\tau_\ell \rightarrow 1$); LAMG+ keeps the same rule with a slightly looser work cap (§3.3). After each cycle the finest iterate is improved by a min-residual (Krylov) recombination over a short history of saved iterates [14, §7.8.2]: $\alpha = \arg \min_\alpha \|r - LE\alpha\|$ with $E = [x_i - x]$, $LE = [r - r_i]$, then $x \leftarrow x + E\alpha$. A level’s history accumulates across the cycle’s γ revisits; the finest persists across cycles.

3. Two refinements beyond LAMG. Reproducing LAMG (§2)—no size cap, orphan singletons, a directional guard, the cycle-index calibration of §3.3—is a prerequisite, not a contribution. LAMG+ departs from LAMG (Table 3.1) by (1) avoiding aggregation across a weak edge, via a strength-of-connection veto (§3.1), and (2) judiciously raising the interpolation caliber from 1 to 2 (§3.2); both target (local) grid-aligned anisotropy—the outlier tail LAMG reported and sketched as future work [1, §5.2, §6.1]. Together they turn a stalled ACF ≈ 0.99 into ≈ 0.11 on the worst anisotropic grids while leaving every other class untouched; the ideas are LAMG’s own, and our contribution is their correct lean realisation and at-scale evidence (§4). Both refinements are *local*—the veto a per-edge test, caliber-2 a per-node test, each firing only where the matrix is locally anisotropic and inert ($\approx 0\%$ of nodes) elsewhere—so refinement follows where the solver is locally slow, never a global average: a graph that is mostly isotropic with a small anisotropic region defeats any global trigger yet is handled correctly, node by node (§3.3). Because the two act at different stages (which nodes aggregate, how they interpolate), they are independent and individually ablatable (§4.7).

TABLE 3.1

The two places LAMG+ departs from LAMG, both targeting grid-aligned anisotropy (the outlier tail LAMG flagged and sketched as future work [1, §6.1]) and both local and self-targeting. Reproduced setup choices (no size cap, orphan singletons, directional guard, non-decaying γ , fixed $K=4$) are in §2, §3.3.

Refinement	LAMG	LAMG+	Effect
Aggregation	affinity + energy guard only	+ strength-of-connection veto: no merge across a weak edge (eq. (3.1))	removes the rare affinity-inversion stall, ACF $0.99 \rightarrow 0.46$
Interpolation caliber	strictly caliber-1	+ selective caliber-2 on locally-1-D nodes	lowers the $q \approx 2$ ceiling, ACF $0.46 \rightarrow 0.11$

3.1. A strength-of-connection veto. Affinity (2.2) ranks a node’s neighbours by how well their relaxed test-vector profiles align, and aggregation merges a node with its highest-affinity admissible neighbour. On a strongly grid-aligned anisotropic grid the test vectors are nearly flat along the weak direction, so the K -sample affinity of a weak (weight- ϵ) edge is computed from almost-identical profiles and can, on an unlucky sample, *exceed* that of the genuinely strong (weight-1) edge—we measure 0.97 versus 0.89 at one interior node of a 256^2 , $\epsilon = 10^{-4}$ grid. Neither the affinity nor the energy guard Q catches this: both are computed from the *same* test vectors and are fooled identically. The *matrix* is not—the conductance ratio there is 10^4 .

A single such inversion creates one cross-weak-direction merge, a localised residual hot-spot (peak-to-mean $> 10^3$), and a stalled rate (ACF ≈ 0.99) that no amount of γ or recombination repairs, and that caliber-2 alone cannot reach past either.

The veto is one line: never aggregate across an edge whose weight is below a fraction τ_{soc} of the node’s strongest incident weight,

$$(3.1) \quad \text{skip edge } (u, v) \text{ in aggregation if } |w_{uv}| < \tau_{\text{soc}} \max_{t \sim u} |w_{ut}|, \quad \tau_{\text{soc}} = 0.05.$$

It costs $\mathcal{O}(1)$ per edge, changes no asymptotics, and is inert wherever a node’s incident weights are comparable. With it, the 256^2 and 384^2 grids drop from ACF ≈ 0.99 to 0.11 across all sizes, seeds, and $\epsilon \leq 10^{-1}$ (worst 0.117 over 24 cases); on a 42-graph random SuiteSparse sample and the social/FE giants to 1.5×10^7 edges it is neutral (worst ACF ratio 1.22, no regression). The defect is rare—it needs a near-degenerate weak direction *and* an unlucky sample—which is why stock LAMG mostly avoids it and reports anisotropy as bounded-but-slow rather than stalled; the veto removes the residual tail and, crucially, is what lets the caliber-2 fit of §3.2 act on a clean aggregation.

3.2. Selective caliber-2 on one-dimensional chains. The veto fixes *which* nodes aggregate; caliber-2 changes *how* they interpolate—the one place LAMG’s strict caliber-1 design leaves a bounded but non-textbook rate. On grid-aligned anisotropy the affinity correctly semicoarsens along the strong direction (the aggregates are well placed), but the caliber-1 piecewise-constant interpolation across each 2:1 aggregate recovers only half of the smooth-in-strong mode, inflating the energy ratio to $q \approx 2$ and pinning the two-level factor at $\rho \approx 0.5$ (App. A). Where a fine node’s *strong* edges reach exactly two coarse aggregates—a locally one-dimensional neighbourhood—LAMG+ adds a second, caliber-2 parent with a single fitted weight w (App. A derives it). Measured per level with an exact coarse solve (so the γ schedule cannot mask it), the two-level factor on the 256^2 , $\epsilon = 10^{-4}$ grid falls from 0.46 (caliber-1, veto on) to 0.28 at every aggregation level; with γ and recombination the full-cycle rate reaches ≈ 0.11 . The gate is *self-targeting*: $\approx 0\%$ of nodes upgrade on isotropic graphs, so over a 1,267-graph SuiteSparse sample it rescues the anisotropic minority (NACA0015, epb3, darcy from 100 cycles to 5–8; the thermomech/thermal stiffness matrices from non-convergence to ≈ 13) while leaving the easy majority’s solve essentially unchanged, at $\approx 1\%$ median wall-clock. It is on by default. The two refinements are complementary: caliber-2 without the veto still stalls at 0.99 on the worst grids, the veto without caliber-2 reaches only the caliber-1 ceiling ≈ 0.46 ; together they give 0.11 (§4.7).

3.3. A single, local configuration. LAMG+ runs one configuration on every graph: a cheap $K=4$ affinity with both refinements always on. The interpolation caliber is raised from 1 to 2 wherever a node is locally one-dimensional (§3.2) and left at caliber-1 everywhere else, so the extra cost falls only on the anisotropic nodes. Four test vectors are an ample sample for the single least-squares weight each upgraded node fits, so we simply fix $K=4$ rather than accumulate test vectors with depth as the original LAMG does (raising K would pay off only together with more relaxation sweeps ν , which we do not pursue).

Because the caliber decision is taken per node, it needs no global control—and that is what makes it robust. A graph that is mostly isotropic with a small anisotropic region is handled correctly: the upgrade fires exactly on that region. Table 3.2 makes the point with a 64^2 anisotropic patch embedded in a 256^2 or 512^2 isotropic grid—only 6% and 1.5% of the nodes, far too few to trip any graph-averaged trigger—where

TABLE 3.2

Caliber-1 vs. the always-on caliber-2 default (ACF, with cycles to relative residual 10^{-8} in parentheses; fixed $K=4$, strength-of-connection veto on). Caliber-2 turns the grid-aligned-anisotropy stall into a healthy rate. “aniso. nodes” is the fraction of locally one-dimensional nodes; on the two mixed grids it is only 6%/1.5%—far too few for any global trigger—yet the per-node upgrade rescues exactly that minority. Non-anisotropic graphs are unaffected (§4.7).

graph	aniso. nodes	caliber-1	caliber-2 (default)
aniso 256^2 , $\epsilon=10^{-4}$	99%	0.56 (13)	0.16 (7)
aniso 384^2 , $\epsilon=10^{-4}$	99%	0.83 (33)	0.11 (6)
isotropic $256^2 + 64^2$ aniso. patch	6%	0.31 (8)	0.02 (5)
isotropic $512^2 + 64^2$ aniso. patch	1.5%	0.24 (7)	0.02 (5)
isotropic grid 256^2	0%	0.016 (5)	0.015 (5)
bodyy5 (FE stiffness)	0%	0.020 (5)	0.019 (5)

caliber-1 stalls (ACF ≈ 0.3) while the always-on local caliber-2 recovers a healthy rate (ACF ≈ 0.02 , five cycles). This is the classical principle that refinement should follow where the solver is *locally* slow, not a global average [14, §9.5].

Cycle index (a reproduction calibration, not a refinement). LAMG’s work rule sets $\gamma_\ell = \min(1.5, 0.7/\tau_\ell)$, floored at 1; LAMG+ uses the same rule with a slightly looser cap and no depth decay,

$$(3.2) \quad \gamma_\ell = \min(1.5 g^{\ell-2}, 0.95/\tau_\ell, 3), \quad \gamma_\ell \geq 1, \quad g = 1.$$

An intermediate state of our port carried a spurious geometric decay ($g = 0.7$) that drove $\gamma \rightarrow 1$ by level four regardless of coarsening; that was a bug, removed here. A rare deep-hierarchy monitor regrows the schedule ($g = 1.15$) if the cycle factor exceeds 0.6; it never fires on any graph or protocol in this paper and exists for the companion flow solver’s source–sink dipole right-hand sides.

4. Numerical results. LAMG+ is implemented in Julia, and all reported times are wall-clock, measured on an Apple M5 Pro (18-core, 48 GB RAM); the competitor solvers of §4.3 run in the same Julia process under identical compilation, so the wall-clock comparisons are language-fair. Throughout this section, every solve uses the right-hand side $b = Lx_{\text{true}}$ with x_{true} a standard-normal random vector projected to zero mean, and runs to relative residual $\|b - Lx\| \leq 10^{-8} \|b\|$ unless a different tolerance is stated. This choice of b exercises the full spectrum through a known solution; right-hand sides concentrated on extreme smooth modes (e.g. a far-separated source–sink dipole) arise in the companion flow paper and are discussed there and in the deep-hierarchy remark of §3.3.

Test corpus. All experiments draw from one fixed corpus, the union of three sources: (i) the complete 1,711-graph SuiteSparse collection used as the LAMG test set ($100 \leq n \leq 1.2 \times 10^7$), which drives the scaling study (§4.2) and, restricted to its $m > 10^6$ members, the head-to-head competition (§4.3); (ii) instances from the approximate-Cholesky robustness benchmark [23], generated by its own generators—all 30 **chimera** and weighted-**chimera** graphs and all 9 uniform/anisotropic/high-contrast Poisson grids—plus our synthesized Kyng–Sachdeva classes (Erdős–Rényi, Barabási–Albert, stochastic-block, k -regular, power-law-cluster), used for robustness (§4.4); and (iii) the genomic de Bruijn giant **kmer_V1r** (2.1×10^8 nodes, 2.3×10^8 edges, 6.8×10^8 nonzeros) for setup scale (§4.2). Every solver runs on the same exported (L, b) ; all per-graph setup and solve times are recorded and released with the code

TABLE 4.1

ACF on identical (L, b) ; “cyc” is cycles to relative residual 10^{-8} . “LAMG+” and “ref.” are the LAMG+ and MATLAB-reference asymptotic convergence factors. Entries $< 10^{-4}$ solve in 1–2 cycles—below the resolution at which an asymptotic factor is meaningful. Class abbreviations: AS = autonomous-systems, P2P = peer-to-peer.

Graph	Class	n	m	LAMG+	cyc	ref.
grid128	structured	16 384	32 512	0.022	5	0.21
bodyy5	FE stiff.	18 589	55 132	0.019	5	0.27
wb-cs-stanford	web	7 008	17 054	0.0009	3	0.06
ca-GrQc	collab.	4 158	13 422	0.0008	3	0.07
ca-HepTh	collab.	8 638	24 806	0.0040	4	0.08
Oregon-1	AS	11 174	23 409	$< 10^{-4}$	2	0.005
as-caida	AS	26 475	53 381	$< 10^{-4}$	2	0.008
p2p-Gnutella08	P2P	6 262	9 719	0.0010	3	0.03
email-Enron	social	33 696	180 811	0.0011	3	0.10
ak2010	road	42 381	102 091	0.0066	4	0.12

so the tables can be regenerated. The largest graph solved end-to-end is `com-Orkut` (1.2×10^8 edges, 2.4×10^8 nonzeros, 2 cycles); on `kmer_V1r` the hierarchy builds but the solve is memory-bound at 48 GB (§4.2).

4.1. Convergence against the reference. Table 4.1 reports the asymptotic convergence factor (ACF, the geometric mean of the last five per-cycle residual reductions) on identical exported systems (L, b) . LAMG+ matches or beats the MATLAB reference on every class. Entries below 10^{-3} are reported to four significant figures: they are small but nonzero, documenting how super-converged the easy classes are.

4.2. Linear scaling. Figure 4.1 plots per-edge setup and solve time versus m for all 1,711 SuiteSparse graphs (the LAMG test set: $50 \leq n \leq 10^6$), spanning 4.3 decades (m from 52 to 9.8×10^5 edges). Every one converges to relative residual 10^{-8} , in a median of 4 cycles. The log-log slopes of per-edge time are $+0.005$ (setup) and -0.002 (solve): per-edge cost is independent of m , the empirical $\mathcal{O}(m)$ statement (equivalently, total setup+solve time scales as $m^{1.01}$). The medians are $t_{\text{setup}}/m \approx 0.24 \mu\text{s/edge}$ (IQR $[0.18, 0.34]$) and $t_{\text{solve}}/m \approx 0.19 \mu\text{s/edge}$ (IQR $[0.10, 0.43]$). These low, flat constants reflect an implementation tuned to the algorithm: a self-gated reverse-Cuthill–McKee reordering for cache locality, single-pass Gustavson-fused Schur extraction (each coarse operator built in one column scan with no matrix-multiply temporary), allocation-free elimination into preallocated buffers, and a monomorphic, branch-free, residual-maintaining in-place Gauss–Seidel kernel; each change is *bit-identical* to a naive implementation—it lowers the constant, not the result. The wide vertical scatter is about a *flat* line—per-instance conditioning (level count, separator quality), not growth with m ; the largest per-edge costs are the strongly anisotropic grids of App. A. Beyond the corpus, the construction scales two orders of magnitude further: on a 2.3×10^8 -edge genomic graph (`kmer_V1r`, 2.1×10^8 nodes) LAMG+ builds its hierarchy at the fixed $K=4$ default—the same cheap configuration it uses everywhere, with no escalation and no probe to misfire. This de Bruijn graph is slow by diameter, not anisotropy; a global trigger keyed on a trial solve would misread that as a stall, whereas the local design never asks the question. The build is a 21-level hierarchy of operator complexity 2.51 in 375 s, the setup remaining $\mathcal{O}(m)$ two decades past the largest LAMG instance. The iterative solve on that one instance

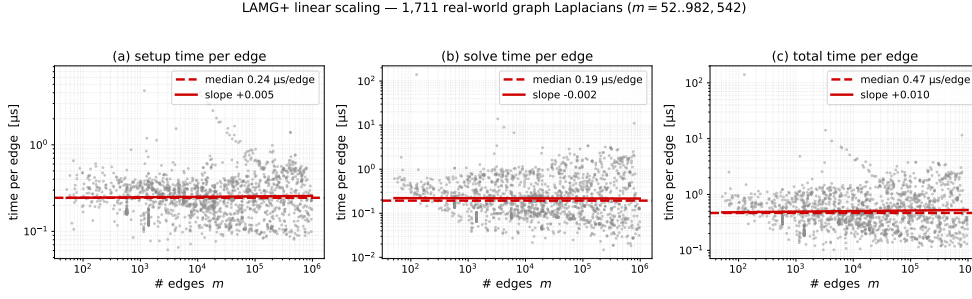


FIG. 4.1. *LAMG+* per-edge time vs. number of edges m (log-log) over 1,711 SuiteSparse graph Laplacians: (a) setup, (b) solve, (c) total. Grey points are individual graphs; the red dashed line is the median, the red solid line the log-log regression fit (slope shown per panel). A near-zero slope means per-edge cost is independent of m — the empirical $\mathcal{O}(m)$ statement; the regression of total setup+solve time gives the exponent $m^{1.01}$.

exceeded our 48 GB: at this scale it is memory-bound, not time-bound.

4.3. Comparison: complementary strengths, by class. We benchmark setup+solve to relative residual 10^{-8} against *both* approximate-Cholesky variants—the original, faster `approxchol_lap` [5] and the worst-case-robust `approxchol_lap2` (“AC”) of [23]—and note sparse Cholesky (CHOLMOD) [15] for context. All run in the same Julia process under identical compilation, so any difference is algorithmic (coarsening vs. fill), not a compiler artifact; we report time per nonzero ($\mu\text{s}/\text{nnz}$, the worst-case metric of [23]), each a warm timed run (a warm-up call first excludes Julia’s one-time JIT compilation), measured serially so the per-graph rows and the aggregate below are contention-free and mutually consistent.

The finding is a clean *class split*, not a single winner. On low-diameter social, web, and citation graphs and on the planar road network, the fast `approxchol_lap` wins: its sampled elimination is cheap where LAMG+’s lean-aggregation setup is not amortised over a 3–4 cycle solve. On finite-element and structural matrices the order reverses—**LAMG+ is the fastest of the three** (geometric-mean $1.76\times$ over the fast `approxchol_lap` across the 34 FE/structural graphs, winning 91% of them; `bmwcra_1`, `crankseg_1`, `troll`, `pwtk` run 1.9 – $3.5\times$ over it), because there setup *is* the dominant cost and the fixed cheap $K=4$ build (§3.3) pays off. Aggregated over all 201 large graphs ($m > 10^6$), the two are *near-peers*: LAMG+ wins 56% of head-to-heads against the fast `approxchol_lap` (geometric-mean $0.99\times$ —`approxChol`’s social/web wins are by similar factors to LAMG+’s FE/structural wins), and 76% against the robust AC, where it is $2.2\times$ faster in the geometric mean; AC’s worst-case robustness costs it $\sim 2\times$ in time. (At the lower 10^{-4} accuracy many applications need, the balance shifts toward LAMG+: §4.5.) Sparse Cholesky is fastest where good separators exist (web crawls, road) but cannot factor the largest poorly-separable matrices at all, so there the contest is purely among the near-linear solvers. Beyond the table, LAMG+ solves `soc-LiveJournal1` (3.3×10^7 edges, 6 cycles), `hollywood-2009` (5.6×10^7 , 3 cycles), and—its largest end-to-end solve—`com-Orkut` (1.2×10^8 edges, 2.4×10^8 nonzeros, 2 cycles), past the largest graph of the original LAMG test set [1].

An a-priori selection rule. We construct a simple predictor of a LAMG+ win from a single statistic computable before either solver runs: the mean degree $\bar{d} = 2m/n$. Treating “LAMG+ wins iff $\bar{d} \geq 30$ ” as a binary classifier over the 201 large graphs,

TABLE 4.2

Setup+solve time per nonzero ($\mu\text{s}/\text{nnz}$) to relative residual 10^{-8} ; bold = fastest. “approxChol” is the fast `approxchol_lap`; “AC” the robust `approxchol_lap2`. Sparse Cholesky is omitted: it is fastest on the web/road graphs but cannot factor the FE/structural and dense-citation matrices (> 21 GB fill on flickr). The bottom block summarises LAMG+ against each competitor over all 201 graphs with $m > 10^6$ ($\text{nnz}(L) \leq 6 \times 10^7$): the geometric-mean speedup is the geometric mean of (competitor time)/(LAMG+ time), so > 1 means LAMG+ is faster; the win rate is the fraction of the 201 graphs on which LAMG+ is faster; each is reported at relative residual $10^{-8}/10^{-4}$. All times are serial (contention-free).

Graph	family	edges	LAMG+	approxChol	AC
web-Stanford	web	1.0 M	0.26	0.19	0.30
web-Google	web	2.6 M	0.48	0.25	0.43
roadNet-PA	road	1.5 M	0.72	0.26	0.34
flickr	social	4.9 M	0.55	0.26	0.62
coPapersDBLP	citation	15.2 M	0.31	0.23	0.46
bmwcra_1	FE	5.2 M	0.08	0.28	0.57
crankseg_1	structural	5.3 M	0.08	0.19	0.65
troll	structural	5.9 M	0.10	0.24	0.44
pwtck	structural	5.7 M	0.11	0.21	0.40
<i>all $m > 10^6$ graphs (201), LAMG+ vs. each ($10^{-8}/10^{-4}$):</i>					
geometric-mean speedup			0.99 / 1.20×		2.20 / 2.91×
win rate			56 / 64%		76 / 84%

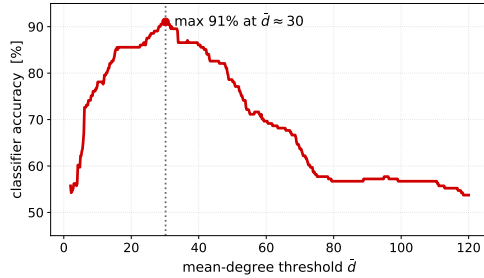


FIG. 4.2. Degree-threshold predictor of a LAMG+ win: accuracy of the rule “LAMG+ wins iff $\bar{d} > t$ ” over the 201 large ($m > 10^6$) graphs as a function of the threshold t . Accuracy peaks at 91% near $\bar{d} \approx 30$ (marked), against a 56% majority-class baseline (the classes are near-balanced: 112 LAMG+ vs. 89 approxChol wins); the point-biserial correlation of $\log \bar{d}$ with a LAMG+ win is $r = +0.68$. The mean degree thus predicts the faster solver a priori.

its accuracy peaks at 91% near $\bar{d} \approx 30$ (Figure 4.2), and the point-biserial correlation of $\log \bar{d}$ with a LAMG+ win is $r = +0.68$. The mechanism is the one behind the FE/structural win: approximate-Cholesky’s fill grows with elimination-clique size, hence with degree, whereas caliber-1 aggregation collapses dense neighbourhoods at $\mathcal{O}(1)$ cost. High mean degree therefore favours LAMG+ and low degree favours sampled elimination, so the appropriate solver can be selected from the degree distribution alone.

Table 4.3 widens the comparison to all seven solvers across twelve graph classes, including all of the approximate-Cholesky benchmark’s synthetic families. The result is a clean separation by design intent. On the structured classes their authors built

TABLE 4.3

Multi-solver comparison by graph class: mean/worst-case setup+solve time per nonzero ($\mu\text{s}/\text{nnz}$) to relative residual 10^{-8} , over a class-stratified corpus (real SuiteSparse/SNAP classes and all of the approximate-Cholesky benchmark’s synthetic families, including the SPE10 reservoir). Bold = fastest mean; a superscript is the number of class instances that did not converge; “DNF” = none converged. A per-graph budget of $50\times$ the fastest solver (capped at 90 s) bounds the slow cases. `apxChol` = `approxchol_lap`; PETSc = PETSc GAMG.

class	LAMG+	apxChol	AC	Boomer	pyAMG	CMG	PETSc
FE/structural	0.12/0.15	0.28/0.54	0.53/0.73	0.18/0.35	0.10 /0.13	0.18/0.21 ²	0.12/0.20
mesh/grid	0.56/1.24	0.25 /0.36	0.68/1.03	1.74/7.02	0.28/0.54	0.64/1.46 ²	0.95/3.47
social/citation	0.47/0.69	0.25 /0.37	0.71/0.94	1.53/2.50	4.60/7.81 ³	0.50/0.73	3.10/3.14 ⁴
web	0.37/0.41	0.29 /0.38	0.39/0.51	1.43/2.59	5.45/5.45 ³	1.27/2.05 ¹	2.76/3.59 ²
road	0.68/0.85	0.39/0.50	0.53/0.65	0.35 /0.39	1.38/1.76	1.24/1.24 ³	0.57/0.75
chimera	0.40/0.91	0.14 /0.19	0.56/1.20	0.48/0.67	1.13/1.74 ¹	0.43/0.78	0.91/1.72
wtd-chimera	0.45/1.34	0.23 /0.70	0.39/0.56	0.46/0.63	2.04/3.25 ¹	0.33/0.76	0.96/1.86
sddm-chimera	0.30/0.51	0.10 /0.13	0.21/0.32	0.44/0.57	2.21/2.80 ¹	0.34/0.47	1.74/3.26
grid-3D	1.30/1.35	0.32/0.37	0.70/0.72	0.19 /0.19	0.37/0.41	DNF	0.25/0.25
aniso-grid	0.61/0.68	0.15/0.16	0.38/0.41	0.15 /0.15	2.38/2.38 ²	1.22/1.48 ¹	1.01/1.49
star	0.05/0.05	0.03/0.03	0.24/0.40	0.02 /0.02	0.03/0.03	0.04/0.06	0.04/0.04
SPE/reservoir	1.09/1.22	0.29 /0.30	0.56/0.59	0.29/0.31	2.72/2.95	DNF	0.90/1.29

them for—FE/structural and grids—the four algebraic-multigrid solvers (LAMG+, pyAMG, BoomerAMG, PETSc GAMG) are all fast and beat sparse Cholesky. On the low-diameter classes—social, citation, web—this reverses: BoomerAMG [30] and PETSc GAMG slow by up to an order of magnitude, pyAMG [29] stops converging, and `approxChol` is fastest. CMG [6] does not converge on the grids, on SPE, or on many roads. Across *all* classes only *three* solvers are both convergent everywhere and never catastrophic (worst case under $1.4\mu\text{s}/\text{nnz}$): **LAMG+** (≤ 1.35), **approxChol** (≤ 0.70), and AC (≤ 1.20)—precisely the robust near-linear solvers this paper compares. BoomerAMG converges everywhere but is catastrophic ($7\mu\text{s}/\text{nnz}$) on irregular graphs; pyAMG, CMG, and PETSc GAMG additionally fail to converge on 11, 15, and 6 of the instances. This is the practical case for LAMG+: not the per-class champion, but—with approximate-Cholesky—one of only two solvers that is never the wrong choice.

Memory. The same split holds in space (Table 4.4). On large FE/structural matrices LAMG+’s no-fill multilevel hierarchy is the *most* frugal of the three (21–30 bytes/nnz vs. 33–55 for the factorisations); on web/social graphs its operators and inter-level transfers outweigh a single Cholesky factor. And across *every* graph the robust AC costs more memory than the fast `approxchol_lap` (its multi-entry splitting), so AC’s guarantee is paid in both time and space. The two AMG competitors store their hierarchies in C, so we compare them by operator complexity $\sum_{\ell} \text{nnz}(A_{\ell})/\text{nnz}(A_0)$, the standard measure: on these FE/structural matrices LAMG+’s gentle $\sim 2\times$ coarsening gives 1.2–1.65, modestly above BoomerAMG (1.02–1.26) and PETSc GAMG (1.00–1.01), whose aggressive coarsening minimises coarse-operator storage—the memory side of the same tradeoff that costs them the robustness of Table 4.3; on the low-diameter classes neither is an appropriate solver. Table 4.4 reports the *persistent* hierarchy; the transient setup peak adds only a *single* level’s K test vectors ($K\cdot n$ floats), discarded once that level is coarsened rather than accumulated across levels, and the fixed $K=4$ default halves this peak (versus LAMG’s $K=8$ accumulation)—which is what governs whether the largest graphs build within a fixed memory budget.

TABLE 4.4

Preconditioner memory (bytes per nonzero of L). LAMG+ is the most frugal on FE/structural matrices, heavier on web/social; AC always exceeds the fast `approxchol_lap`.

Graph	LAMG+	approxChol	AC
troll, pwtk (structural)	23	34	44
bmwcra_1 (FE)	22	35	47
bone010 (FE, 2.3×10^7)	30	43	55
flickr (social)	80	40	53
coPapersDBLP (citation)	49	32	32
web-Stanford (web)	62	32	32

4.4. Robustness across Spielman’s test families. The crossover above is a speed result; robustness—converging at all—is a separate axis, and the one on which graph-Laplacian solvers most often differ. Gao, Kyng, and Spielman [23] report AC as the only solver in their study (alongside CMG, hypre/BoomerAMG, PETSc, and ICC) to converge across their full family set—random `chimera` graphs and randomly-weighted variants, SDDM “chimeras”, adversarial “Sachdeva stars”, an SPE benchmark, and uniform/anisotropic/high-contrast Poisson grids. That robustness is precisely why we take AC as the near-linear baseline worth a same-machine comparison.

LAMG+ meets the same bar. With the standard right-hand side ($b = Lx_{\text{true}}$, zero-mean), it converges to *both* 10^{-8} and 10^{-4} on every one of the benchmark’s families generated by Spielman’s own `Laplacians.jl`: `chimeras` and their weighted, uniform, and semi-weighted variants (96 graphs, $n = 10^4$ to 10^6), SDDM chimeras (12), 3-D Poisson grids with uniform, anisotropic, and high-contrast coefficients (12), and Sachdeva-style stars (3). It also converges on the benchmark’s one *external* family, the SPE fluid dataset: we reconstruct the SPE10 reservoir model [24] directly from its public permeability field, discretised by a two-point flux approximation into high-contrast (permeability ratio $\sim 10^7$), anisotropic graph Laplacians at three sizes ($n = 2.6 \times 10^5$ to 1.1×10^6), each solved in 5 cycles. Extending *past* the scale at which AC was benchmarked, it further solves 41 large SuiteSparse graphs up to `com-Orkut` (1.2×10^8 edges, 2.4×10^8 nonzeros). Together with the 1,711-graph scaling set (§4.2), every instance converges, in a median of 4 cycles. The original MATLAB LAMG was reported non-convergent on parts of this set [23]; our independent re-derivation converges on all of them—indicating the failure was specific to that implementation, not the algorithm—which required matching the original’s design choices (§2) line by line.

On these well-structured families approximate-Cholesky is faster—bounded degrees and separators suit its sampled elimination—while LAMG+’s advantage is the large poorly-separable class of Table 4.2. Both are robust, so the comparison is one of speed, reported next over the full corpus in time per nonzero (the worst-case metric of [23]).

4.5. Lower accuracy shifts the balance toward LAMG+. Many applications need only a few digits. At relative residual 10^{-4} the LAMG+ solve collapses to a *single* cycle, so the solve’s share of the total shrinks and setup—the phase where LAMG+’s cheap fixed- $K=4$ build is strongest—dominates. Lowering the tolerance therefore amplifies whichever solver wins on setup, and that helps LAMG+ across the board (Table 4.2, bottom block). Against the fast `approxchol_lap` the aggre-

TABLE 4.5

When each choice is necessary, and its effect when wrong (negligible elsewhere). “Stresses” is the graph class on which the choice is load-bearing. Top: LAMG choices LAMG+ reproduces (§2, §3.3); bottom: the two anisotropy refinements.

Choice	Stresses	Effect if wrong
no size cap	scale-free / web / social	web ACF $0.002 \rightarrow 0.97$ (dense fill, stall)
orphan singletons	anisotropic FE	bodyy5 diverges ($\rightarrow 0.99$)
directional guard	structured	over-aggregation; rarely fatal alone
non-decaying γ ($g=1$)	deep hierarchies	a spurious $g=0.7$ decay starves recombination
strength-of-connection veto	grid-aligned anisotropy	rare affinity-inversion stall ($\rightarrow 0.99$)
caliber-2 (default on)	grid-aligned anisotropy	ceiling $0.46 \rightarrow 0.11$; 100-cyc near-failures $\rightarrow 5-14$

gate geometric mean rises from $0.99\times$ at 10^{-8} (parity) to $1.20\times$ at 10^{-4} —LAMG+ ahead—and its win rate over the 201 large graphs rises from 56% to 64%. Against the robust AC the lead widens from $2.20\times$ to $2.91\times$ ($76\% \rightarrow 84\%$ of head-to-heads). The shift is largest exactly where LAMG+ already leads, the FE/structural class, since there the solve was already short and setup already dominant.

4.6. Anisotropy: the veto and the caliber-2 extension. Grid-aligned anisotropy is baseline (caliber-1) LAMG+’s worst class. On a 64×64 grid with strong weight 1 and weak weight ϵ the full solver converges at a bounded but non-textbook ACF $\approx 0.27-0.39$ for $\epsilon \leq 10^{-1}$ (vs. 0.01 isotropic); the caliber-2 gate (§3.2) recovers a near-geometric rate, ACF = $0.007/0.05/0.10$ at $\epsilon = 10^{-1}/10^{-2}/10^{-4}$, at lower operator complexity (sharper interpolation buys fewer levels). On larger grids (256^2 , 384^2) a second, rarer failure appears: the affinity inversion of §3.1 produces a single bad merge that stalls even caliber-2 at ACF ≈ 0.99 . The strength-of-connection veto removes it, and the two refinements together hold ACF ≈ 0.11 across all sizes, seeds, and $\epsilon \leq 10^{-1}$ (worst 0.117 over 24 cases). On the SuiteSparse stiffness matrices the pair converts 100-cycle near-failures into 5–14 cycle solves. This does *not* make LAMG+ the fastest solver on this class—these matrices are small and well-separated, so `approxChol` and `CHOLMOD` stay ahead—so the value here is robustness, closing LAMG’s documented outlier tail, not a new crossover. The mechanism, the per-level two-level factors, and the weight derivation are in App. A.

4.7. Ablation Study. Each choice—the reproduced LAMG choices, the cycle-index calibration, and the two refinements of §3—is load-bearing only on the class that stresses it. Table 4.5 ablates each singly. The dominant setup cost is the test-vector smoothing, fixed at the cheap $K=4$ everywhere (§3.3); the anisotropy fix comes from caliber-2, not from more test vectors (Table 3.2). The veto and caliber-2 are *complementary* on their target class: on the 256^2 , $\epsilon = 10^{-4}$ grid the veto alone reaches its caliber-1 ceiling (ACF ≈ 0.46), caliber-2 alone still stalls at 0.99 (the affinity inversion survives), and only both together give 0.11.

5. Toward a parallel LAMG+. The results above are single-threaded. Both hot phases parallelize in principle—the setup (aggregation, test-vector relaxation, Galerkin triple products) and the cycle reduce to the scalable sparse primitives (sparse matrix–vector products, maximal independent set, segmented reductions) that Bell, Dalton, and Olson [32] exposed for GPU AMG and that scale BoomerAMG to 10^5 cores [33, 34]—but the one phase that *looks* inherently sequential is the Gauss–Seidel smoother, since each update reads its already-updated neighbours. We quantify that it is not, then state the obstacle that remains open.

The smoother survives parallelization (a smoothing-factor analysis). The standard parallel substitute for lexicographic GS is *block-Jacobi GS*: partition the nodes into P contiguous blocks, each of width $W=n/P$, of a bandwidth-reducing reverse Cuthill–McKee (RCM) ordering [17]; run serial GS within each block and freeze (Jacobi) the cross-block coupling, so the blocks relax concurrently. In the RCM order write $A = D - L - L^\top$; serial GS uses the preconditioner $M_{\text{GS}} = D - L$, and block-Jacobi GS its block-diagonal restriction $M_{\text{BJ}} = D - L_{\text{blk}}$. A relaxation smooths iff it contracts the high-frequency error; the smoothing factor is $\mu = \|P_{\text{HF}} S P_{\text{HF}}\|_2$, where $S = I - M^{-1}A$ and P_{HF} is the orthogonal projector onto the eigenvectors of A above its median eigenvalue (on a regular grid, the upper Fourier quadrant). The reference values are $\mu=1$ (undamped Jacobi, no smoothing), $\mu_{\text{GS}} \approx \frac{1}{2}$, and $\mu_{\text{RB}} = \frac{1}{4}$ (red–black).

PROPOSITION 5.1. *Let $\sigma = (\sum_{\text{severed}} w_{uv}) / (\sum_{(u,v) \in E} w_{uv})$ be the fraction of edge weight cut by the block boundaries. Then $\mu_{\text{BJ}} = \mu_{\text{GS}} + O(\sigma)$, and for width- W blocks of a d -dimensional grid $\sigma = O(1/W)$. Hence $\mu_{\text{BJ}} \rightarrow \mu_{\text{GS}}$ as $W \rightarrow \infty$, reaching the Jacobi value $\mu=1$ only at $W=1$.*

Proof. $E := M_{\text{GS}} - M_{\text{BJ}}$ collects exactly the severed strict-lower couplings, so $\|E\| = O(\sigma\|A\|)$. Then $S_{\text{BJ}} - S_{\text{GS}} = (M_{\text{GS}}^{-1} - M_{\text{BJ}}^{-1})A = -M_{\text{GS}}^{-1}E M_{\text{BJ}}^{-1}A$, whose prefactors are bounded on the high-frequency subspace, so $\|P_{\text{HF}}(S_{\text{BJ}} - S_{\text{GS}})P_{\text{HF}}\| = O(\sigma)$ and the estimate follows. On a grid of side L cut into $P=L/W$ slabs, the $P-1$ interfaces carry $O(L^{d-1})$ edges each of $O(L^d)$ total, whence $\sigma = O(P/L) = O(1/W)$. $\square$

Three independent measurements—the operator norm, local Fourier analysis (grids only), and direct high-frequency sweeps—agree where each applies and bear out Proposition 5.1: at the operative $P=8$, μ_{BJ} matches μ_{GS} to within a few percent on grids and social graphs alike, reaching the Jacobi limit only as $W \rightarrow 1$ (Table 5.1). The estimate also predicts the non-obvious case: as the penalty scales with severed *weight*, an RCM-banded mesh (few but heavy interface edges) degrades more than a scale-free graph (many but light), as the table shows.

The smoothing factor governs the smoother alone. A block-Jacobi *cycle* can still converge more slowly with μ unchanged: the frozen seams act as $\omega=1$ Jacobi in the inter-block transfer and degrade the coarse-grid correction—a separate effect, invisible to μ and to a cycle count, and absent from a multicolour smoother that recovers rather than freezes the coupling. Brandt’s prescription is accordingly to *colour, not freeze* (red–black/zebra GS are fully parallel and smooth better, $\mu=\frac{1}{4}$, at lower cost [14]). Proposition 5.1 makes the narrower point that suffices here: the LAMG+ smoother has no *intrinsic* serial bottleneck—its parallel form loses no smoothing power to leading order—unlike the global triangular solve of an approximate-Cholesky factor (§4.3), whose elimination order has no comparably local form.

The open obstacle is distributed coarsening. An irregular, low-diameter Laplacian must be partitioned to minimise inter-processor communication—itsself a multi-level graph-partitioning problem [35], handled in production distributed AMG (hypr,

TABLE 5.1

Block-Jacobi vs serial GS smoothing factor $\mu = \|P_{\text{HF}}SP_{\text{HF}}\|_2$ (operator norm), over P contiguous RCM blocks. At operative block counts $\mu_{\text{BJ}} \approx \mu_{\text{GS}}$; only at the extreme $P=n$ (one node per block) does it reach the pure-Jacobi non-smoother $\mu=1$. The cut-edge fraction does not predict the penalty: the bounded-degree mesh degrades more than the scale-free graph despite far fewer cuts.

case	P	cut frac.	μ_{GS}	μ_{BJ}
32×32 model-problem grid	8	—	0.424	0.437
32×32 model-problem grid	n	—	0.424	1.000
netz4504 (bounded-degree mesh)	8	0.12	0.48	0.73
Arenas-email (scale-free)	8	0.73	0.49	0.55

PETSc GAMG) by repartitioning each coarse level with ParMETIS-style tools—and aggregates straddling a boundary need a per-cycle halo exchange whose cost is set by the highest-degree boundary nodes. LAMG+’s coarsening is, however, local: the per-node affinity and caliber-2 tests read only a node’s immediate neighbourhood, so they distribute naturally once a partition is fixed. Whether that locality preserves LAMG+’s lean operator complexity under a communication-minimising partition is a project in its own right, beyond this paper’s scope. We note only that nothing in the algorithm is inherently sequential—a no-fill aggregation hierarchy is, if anything, friendlier to distribution than a sparsified Cholesky factor whose elimination order is global.

6. Conclusion. LAMG+ is a lean, parameter-free, empirically linear-time graph-Laplacian solver. Its contribution is threefold. *Robustness*: a faithful reproduction of LAMG’s load-bearing setup choices (§2) plus two targeted, *local* refinements (§3)—a strength-of-connection veto and a selective caliber-2 interpolation—make one algorithm converge to 10^{-8} across structured, web, social, citation, collaboration, road, and grid-aligned anisotropic graphs with no per-graph tuning, and on every family of Spielman’s approximate-Cholesky benchmark [23], including those on which the original MATLAB LAMG was reported non-convergent. *Scale*: empirical $\mathcal{O}(m)$ over 1,711 SuiteSparse graphs (100% converged, median 4 cycles) and an $\mathcal{O}(m)$ hierarchy built on a 2.3×10^8 -edge graph. *A fair adjudication of two paradigms*: run in the same language, energy-corrected aggregation and approximate-Cholesky are peers with complementary strengths—approximate-Cholesky faster on social and citation graphs, LAMG+ the fastest *and* most memory-frugal solver on finite-element/structural matrices, and $2.2\times$ faster on large graphs than the worst-case-robust approximate-Cholesky variant—while sparse Cholesky cannot factor the largest poorly-separable matrices. Both refinements are local and self-targeting, which is what lets LAMG+ stay cheap by default—a single fixed configuration, with the anisotropy fix applied node by node—yet robust on demand; a full distributed parallelization (§5) is open—we establish only that the smoother, often a multigrid solver’s sequential bottleneck, admits a parallel block-Jacobi ordering that preserves its smoothing factor, whereas a sparsified-Cholesky factor’s global elimination order has no comparably local form. Being a multigrid rather than a factorization, LAMG+’s caliber-1 aggregation, fractional cycle, and recombination also reuse within a Full-Approximation-Scheme solver [8] for the nonlinear (max-flow) companion paper.

Appendix A. Grid-aligned anisotropy, and the caliber-2 fix.

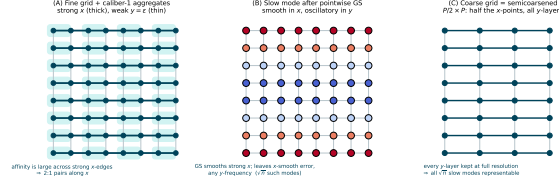


FIG. A.1. Why caliber-2 is a one-dimensional fix. (A) Affinity merges along the strong direction (2:1 x -pairs)—a semicoarsening. (B) The error pointwise GS leaves is smooth in x , oscillatory in y . (C) The semicoarsened grid keeps every y -layer; caliber-1 interpolates the smooth- x mode across each x -pair by a constant ($q \approx 2$, $\rho \approx 0.5$), caliber-2 by a line ($q \rightarrow 1$, $\rho \approx 0.12$).

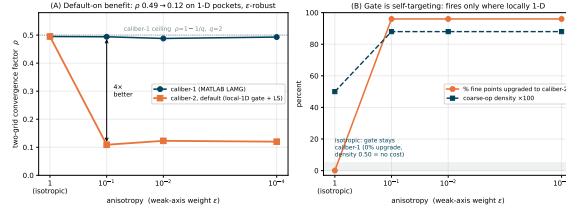


FIG. A.2. The caliber-2 fix (§3.2, on by default), two-grid factor on an anisotropic grid. (A) On locally one-dimensional pockets the second parent drops ρ from the caliber-1 ceiling ≈ 0.49 to ≈ 0.12 , ϵ -robust. (B) The gate is self-targeting: $\approx 96\%$ of fine nodes upgrade where the neighbourhood is 1-D and $\approx 0\%$ on the isotropic grid.

On a five-point grid with strong (horizontal) weight 1 and weak (vertical) weight ϵ , the headline inverts the naive expectation: an isotropic grid is easy (ACF = 0.01), heterogeneous weights (each edge an independent random factor over three decades) are also easy (ACF = 0.006), but *grid-aligned* anisotropy is the hard case (ACF ≈ 0.34 – 0.39 at $\epsilon \leq 10^{-2}$). The mechanism and the cure are both in the LAMG design. Affinity *does* semicoarsen—it merges along the strong direction, the right *kind* of coarsening. The difficulty is neither the smoother nor the coarsening direction but the caliber-1 piecewise-constant interpolation: it recovers about half the magnitude of a smooth-in-strong mode, inflating the coarse-operator energy by $q \approx 2$ and pinning the *two-level* factor at $\rho \approx 1 - 1/q \approx 0.5$ [1, §3.4.3]. This is an interpolation-order property, governed by the classical rule that the interpolation order plus the residual-transfer order must reach the order of the operator, $m_i + m_j \geq m_{ij}$ [14, §4.3]: caliber-1 reproduces constants but not the linear-in-strong mode, so it is order-deficient for the $m_{ij}=2$ Laplacian along the strong chain. The scalar q is the *symptom*; the cause is order, and the number of coarse parents needed to restore it—the *caliber*—equals the dimension of the locally-smooth subspace [1, §3.1.2]. A strong-direction line smoother would only double-cover the mode the semicoarsening already handles, and larger aggregates do not help, since ρ is set by q , not size.

The order deficit recurs identically on every semicoarsened level, so left uncorrected it *compounds*: a flat V-cycle ($\gamma = 1$, no recombination) sits at ACF ≈ 0.77 —the multilevel rate the deficit accumulates to, not the two-level ≈ 0.5 . The fractional cycle index $\gamma = 1.5$ and iterate recombination at every level [1, §3.4.5] *bound* this compounding (Table A.1): with both on, the $\epsilon = 10^{-4}$ grid converges at ACF = 0.39 (15 cycles), and recombination off leaves the stationary 0.51. But recombination minimises only over the span of saved smoothed iterates; it cannot drive an order-deficient interpolation to a geometric rate, because the same deficiency reappears at each level.

570 Only raising the interpolation order (caliber-2) removes it. So at caliber-1 aligned
 571 anisotropy is robust but not textbook—a bounded ≈ 0.4 , the correct lean-algebraic
 572 trade.

TABLE A.1

Controlled anisotropy on a 64×64 grid, full LAMG+ solver. The two-level ceiling ($\rho \approx 0.5$, caliber-1 energy inflation) is bounded into a robust multilevel rate by recombination + $\gamma = 1.5$; a flat V-cycle exposes the raw ceiling. Caliber-2 recovers a near-geometric rate. ACF = asymptotic convergence factor; cyc = cycles to 10^{-10} .

weak weight ϵ	caliber-1 ($\gamma=1.5$ +recomb.)	caliber-2	flat V-cycle
1 (isotropic)	0.010 (5 cyc)	0.008 (4 cyc)	0.74 (40 cyc)
10^{-1}	0.274 (11 cyc)	0.007 (4 cyc)	0.76 (44 cyc)
10^{-2}	0.335 (14 cyc)	0.051 (5 cyc)	0.77 (53 cyc)
10^{-4}	0.391 (15 cyc)	0.100 (6 cyc)	0.77 (56 cyc)

573 *The affinity-inversion defect and the strength-of-connection veto.* Table A.1 is a
 574 64^2 grid, small enough that aggregation never misfires. On larger grids (256^2 , 384^2) a
 575 rarer failure surfaces, and it must be fixed before caliber-2 can act. The affinity (2.2)
 576 is a K -sample squared cosine of relaxed test-vector profiles; along the *weak* direction
 577 those profiles are nearly identical, so the weak edge’s sampled affinity has high variance
 578 and can, at an unlucky interior node, *exceed* the strong edge’s—we measure 0.97
 579 (weak, weight $\epsilon = 10^{-4}$) against 0.89 (strong, weight 1) at one node. The energy
 580 guard Q shares the same test vectors and is fooled identically; only the matrix, with
 581 its 10^4 conductance ratio, is not. The resulting single cross-weak merge is a localised
 582 defect—one size-4 aggregate spanning the weak direction amid a sea of correct 2:1
 583 strong-direction pairs—a weakly-connected aggregate with a *very large* local energy
 584 ratio [1, §3.4.2]. It is the *same* law $\rho \approx 1 - 1/q$ that gives the caliber-1 ceiling, but
 585 now at $q \gg 1$ rather than $q \approx 2$: one bad aggregate pins a residual hot-spot (peak-
 586 to-mean $> 10^3$) and a stalled ACF ≈ 0.99 that neither γ nor caliber-2 can pass—and
 587 recombination cannot either, since the defective coarse operator reintroduces the bad
 588 mode each cycle, outside the smoothed-iterate span recombination acts on. The two
 589 regimes of one law thus take two cures: the veto for $q \gg 1$, caliber-2 for $q \approx 2$. The
 590 veto (3.1) removes it at $\mathcal{O}(1)$ per edge by refusing any merge across an edge below
 591 $\tau_{\text{soc}} = 0.05$ times the node’s strongest incident weight. With the veto on, the per-
 592 level two-level factor (exact coarse solve, $\gamma = 1$, so the cycle schedule cannot mask the
 593 interpolation) is ≈ 0.46 at caliber-1 and ≈ 0.28 at caliber-2 across every aggregation
 594 level of the 256^2 grid; the full solver then reaches ACF ≈ 0.11 . The two refinements
 595 are thus sequential: the veto restores a clean semicoarsening, on which the caliber-2
 596 fit below is well posed.

597 *Fitting the caliber-2 weight.* A fine node u whose strong edges reach exactly two
 598 coarse aggregates a, b is interpolated by $\phi_u = w \phi_a + (1 - w) \phi_b$; fixing the weights
 599 to sum to 1 keeps constants exact. The single weight w is fit by least squares over
 600 the *same* K test vectors used for affinity (no extra setup): minimise $\sum_k \omega_k (x_u^{(k)} -$
 601 $[w x_a^{(k)} + (1 - w) x_b^{(k)}])^2$, whose closed form, with $d_k = x_a^{(k)} - x_b^{(k)}$ and $e_k = x_u^{(k)} - x_b^{(k)}$,
 602 is

603 (A.1)
$$w^* = \frac{\sum_k \omega_k d_k e_k}{\sum_k \omega_k d_k^2},$$

the regression of u 's profile onto the strong chain's endpoints. The smoothness weights ω_k down-weight test vectors with large residual energy at u . We guard $w^* \in [0, 1]$ (a convex combination, so the Galerkin operator gains no spurious anti-coupling), snapping near-boundary values to 0/1; an out-of-range fit signals that u is not truly one-dimensional and is left caliber-1. The gate fires only when the two strong-reachable aggregates are distinct and the fit is admissible—self-targeting at no parameter cost. A seed-injection restriction R with $R \cdot P = I$ (each coarse node injects from its seed) makes the operator Full-Approximation-Scheme-safe, carrying the extension to the nonlinear companion solver.

The role of caliber-2. The guard Q already makes the solver robust in every dimension ($\rho \lesssim 1 - 1/Q$, never diverges). What it does not do is make q *small* where anisotropy is persistent: grid-aligned anisotropy and 1-D FE chains sit at the ceiling $q \approx 2$ at every level. Caliber-2 attacks that residue, and only where two conditions coincide: the bound is tight (the strong coupling is a line, so caliber-1 across it is exactly the half-recovered mode) and the cure is cheap (two collinear parents). Both hold iff a node is locally one-dimensional. Hence anisotropic 2-D and 3-D *rods* (a single strong direction) become textbook, while 3-D *plates* (strong coupling in a plane) are locally two-dimensional: the gate sees > 2 strong aggregates and correctly declines, since recovering textbook there needs caliber-3 (three non-collinear parents), unconstructible in the lean design and left to the guard. The division of labour is clean: the guard buys universal robustness; caliber-2 buys textbook efficiency in the one regime where the bound is tight and the fix costs a single parent.

Appendix B. Linear-time evaluation of the energy guard. A direct evaluation of (2.3) is superlinear: recomputing, for each candidate (u, s) pair, the nodal energy by summing over u 's incident edges for each of K test vectors costs $\mathcal{O}(K \deg u)$ per pair, and a node with D admissible neighbours is tested $\mathcal{O}(D)$ times, so the work is $\mathcal{O}(K \deg^2)$ —concentrated on the high-degree hubs that dominate the total. The guard admits an $\mathcal{O}(K)$ -per-candidate form. In one $\mathcal{O}(Km)$ pass accumulate, per node u , the three moments $s_0 = \sum_v w_{uv}$, $s_1^{(k)} = \sum_v w_{uv} x_v^{(k)}$, $s_2^{(k)} = \sum_v w_{uv} (x_v^{(k)})^2$. Then both energies in (2.3) are closed forms: the relaxed (optimal) energy is $\min_y E_u = \frac{1}{2}(s_2^{(k)} - (s_1^{(k)})^2/s_0)$ and the forced energy is $E_u(\cdot; x_s) = \frac{1}{2}(x_s^2 s_0 - 2x_s s_1^{(k)} + s_2^{(k)})$, so each candidate costs $\mathcal{O}(K)$, independent of degree. Only the *cost* changes, not the value: every aggregation decision, hierarchy, and ACF is identical to a naive evaluation. This holds the setup at the flat per-edge cost of Fig. 4.1 (regression slope 0.008).

Appendix C. Validation methodology. The implementation was validated line-by-line against the unmodified MATLAB reference `lamg-2.2.1` run under Octave: we recompiled its C MEX kernels (so Octave executes the *exact* Gauss–Seidel, Galerkin, aggregation-sweep, and low-degree-sweep kernels) and mechanically adapted Octave's class incompatibilities. We exported each (L, b) from the implementation and loaded the identical system in Octave, eliminating preprocessing as a variable; both emit a state trace (level, residual norm, iterate norm) in the same format. We verified that the elimination set $|F|$, the Schur operators (residual 10^{-16}), and cycle stationarity (recombination off $\Rightarrow$ constant asymptotic factor) match, and that the reference performs no weight preprocessing.

REFERENCES

- [1] O. E. Livne and A. Brandt, *Lean Algebraic Multigrid (LAMG): Fast Graph Laplacian Linear Solver*, SIAM J. Sci. Comput. **34**(4), B499–B522, 2012.
- [2] D. A. Spielman, *Algorithms, Graph Theory, and Linear Equations in Laplacian Matrices*, Proc. ICM, 2010.
- [3] D. A. Spielman and S.-H. Teng, *Nearly-linear time algorithms for graph partitioning, graph sparsification, and solving linear systems*, SIAM J. Comput. **40**(4), 2011.
- [4] J. A. Kelner, L. Orecchia, A. Sidford, and Z. A. Zhu, *A simple, combinatorial algorithm for solving SDD systems in nearly-linear time*, STOC, 2013.
- [5] D. A. Spielman et al., *Laplacians.jl (approxChol)*; R. Kyng and S. Sachdeva, *Approximate Gaussian elimination for Laplacians*, FOCS, 2016.
- [6] I. Koutis, G. L. Miller, and D. Tolliver, *Combinatorial preconditioners and multilevel solvers for problems in computer vision and image processing*, Comput. Vis. Image Underst. **115**(12), 2011.
- [7] A. Brandt, S. F. McCormick, and J. Ruge, *Algebraic multigrid (AMG) for sparse matrix equations*, in *Sparsity and its Applications*, Cambridge, 1984.
- [8] A. Brandt and O. E. Livne, *Multigrid Techniques: 1984 Guide with Applications to Fluid Dynamics, Revised Edition*, SIAM, 2011 (Full Approximation Scheme, nonlinear multigrid).
- [9] J. W. Ruge and K. Stüben, *Algebraic multigrid*, in *Multigrid Methods*, SIAM Frontiers in Appl. Math. **3**, 1987.
- [10] P. Vaněk, J. Mandel, and M. Brezina, *Algebraic multigrid by smoothed aggregation for second and fourth order elliptic problems*, Computing **56**, 1996.
- [11] Y. Notay, *An aggregation-based algebraic multigrid method*, ETNA **37**, 2010.
- [12] A. Brandt, J. Brannick, K. Kahl, and I. Livshits, *Bootstrap AMG*, SIAM J. Sci. Comput. **33**(2), 2011.
- [13] D. Ron, I. Safro, and A. Brandt, *Relaxation-based coarsening and multiscale graph organization*, Multiscale Model. Simul. **9**(1), 2011.
- [14] A. Brandt, *Multigrid Techniques: 1984 Guide with Applications to Fluid Dynamics*, GMD-Studie 85, 1984; reissued SIAM, 2011.
- [15] T. A. Davis, *Direct Methods for Sparse Linear Systems*, SIAM, 2006 (CHOLMOD).
- [16] A. George, *Nested dissection of a regular finite element mesh*, SIAM J. Numer. Anal. **10**(2), 1973.
- [17] E. Cuthill and J. McKee, *Reducing the bandwidth of sparse symmetric matrices*, in Proc. 24th National Conference of the ACM, 1969, pp. 157–172.
- [18] R. J. Lipton, D. J. Rose, and R. E. Tarjan, *Generalized nested dissection*, SIAM J. Numer. Anal. **16**(2), 1979.
- [19] E. G. Boman, B. Hendrickson, and S. Vavasis, *Solving elliptic finite element systems in near-linear time with support preconditioners*, SIAM J. Numer. Anal. **46**(6), 2008.
- [20] S. I. Daitch and D. A. Spielman, *Faster approximate lossy generalized flow via interior point algorithms*, STOC, 2008.
- [21] A. Y. Ng, M. I. Jordan, and Y. Weiss, *On spectral clustering: analysis and an algorithm*, NeurIPS, 2002.
- [22] M. Fiedler, *Algebraic connectivity of graphs*, Czechoslovak Math. J. **23**, 1973.
- [23] Y. Gao, R. Kyng, and D. A. Spielman, *Robust and practical solution of Laplacian equations by approximate elimination*, arXiv:2303.00709, 2023 (the `approxchol_lap2` solver in `Laplacians.jl`).
- [24] M. A. Christie and M. J. Blunt, *Tenth SPE comparative solution project: a comparison of upscaling techniques*, SPE Reservoir Eval. & Eng. **4**(4), 308–317, 2001 (the SPE10 benchmark; public permeability field).
- [25] A. Napov and Y. Notay, *An efficient multigrid method for graph Laplacian systems*, ETNA **45**, 2016; and ... II: *Robust aggregation*, SIAM J. Sci. Comput. **39**(5),

- 2017.
- [26] J. Brannick, Y. Chen, J. Kraus, and L. Zikatanov, *Algebraic multilevel preconditioners for the graph Laplacian based on matching in graphs*, SIAM J. Numer. Anal. **51**(3), 2013.
- [27] B. Lee, *Bringing physics into the coarse-grid selection: approximate diffusion-distance/effective-resistance measures for network analysis and algebraic multigrid for graph Laplacians*, Numer. Linear Algebra Appl. **31**(2), 2024.
- [28] C. Chen, T. Liang, and G. Biros, *RCHOL: randomized Cholesky factorization for solving SDD linear systems*, SIAM J. Sci. Comput. **43**(6), 2021.
- [29] N. Bell, L. N. Olson, J. Schroder, and B. Southworth, *PyAMG: algebraic multigrid solvers in Python*, J. Open Source Softw. **8**(87), 2023.
- [30] V. E. Henson and U. M. Yang, *BoomerAMG: a parallel algebraic multigrid solver and preconditioner*, Appl. Numer. Math. **41**(1), 2002.
- [31] I. Luz, M. Galun, H. Maron, R. Basri, and I. Yavneh, *Learning algebraic multigrid using graph neural networks*, ICML, 2020.
- [32] N. Bell, S. Dalton, and L. N. Olson, *Exposing fine-grained parallelism in algebraic multigrid methods*, SIAM J. Sci. Comput. **34**(4), C123–C152, 2012.
- [33] A. H. Baker, R. D. Falgout, T. V. Kolev, and U. M. Yang, *Scaling hypre’s multigrid solvers to 100,000 cores*, in *High-Performance Scientific Computing*, Springer, 2012.
- [34] M. Naumov et al., *AmgX: a library for GPU accelerated algebraic multigrid and preconditioned iterative methods*, SIAM J. Sci. Comput. **37**(5), S602–S626, 2015.
- [35] G. Karypis and V. Kumar, *A fast and high quality multilevel scheme for partitioning irregular graphs*, SIAM J. Sci. Comput. **20**(1), 359–392, 1998.